

Angular

Introduction to Angular • Setting up an Angular Application Components, Modules & Services • Data Binding & Templates Forms & Validation • Routing & Navigation HTTP & Observables • Building a Complete MEAN Stack Frontend

1. Introduction to Angular

Angular is an open-source web application framework maintained by Google and a community of developers. It is designed to build dynamic and interactive single-page applications (SPAs) efficiently.

Angular was initially released in 2010 by Google. The first version, **AngularJS**, introduced concepts like two-way data binding and directives. In 2016, **Angular 2** was released as a complete rewrite of AngularJS, focusing on modularity and performance. Since then, Angular has continued to evolve with regular updates to meet the demands of modern web development.

Key Features of Angular

Components — Angular breaks your app into small reusable pieces called components. Each piece controls one part of the screen (like a navbar, card, or button).

Data Binding — Angular automatically connects your data (TypeScript) to your HTML. When data changes, the screen updates on its own — no manual work needed.

Directives — Special commands you add in HTML to control behaviour. For example, **ngIf* shows/hides elements, and **ngFor* repeats a list automatically.

Dependency Injection — Angular automatically provides services (like data fetchers) to components that need them. You don't have to create them manually every time.

Routing — Angular lets users move between different pages without reloading the browser. It feels fast and smooth like a mobile app.

Forms — Angular has powerful built-in tools to handle user input forms — with easy validation (like checking if an email is correct).

HTTP Client — Angular has a built-in tool to talk to servers and fetch data from APIs easily (like getting a list of products from a backend).

TypeScript Support — Angular is built with TypeScript, which catches errors before the app even runs — making your code more reliable and easier to manage.

2. Setting Up an Angular Application

Step 1 — Install Angular CLI

Angular CLI (Command Line Interface) is a powerful tool for creating and managing Angular applications.

```
npm install -g @angular/cli
```

Step 2 — Create a New Angular Project

Use Angular CLI to create a new Angular project. Navigate to the desired directory and run:

```
ng new my-angular-app  
cd my-angular-app
```

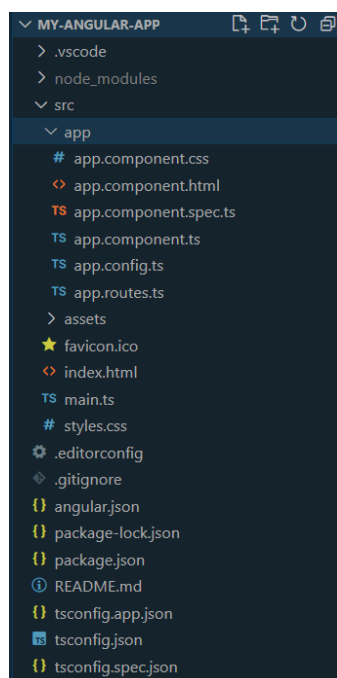
Step 3 — Serve the Application

Launch the development server to see your app in action:

```
ng serve
```

Folder Structure

After creating the project you will see the following folder structure. The **src/app** directory is where you write most of your application code.



Angular project folder structure in VS Code

How Angular Renders Your App

Understanding how Angular turns your code into what you see in the browser is essential. Here is the step-by-step flow:

Step 1 — Write your template

In *app.component.html* you write normal HTML markup: `<h1>Hello World</h1>`

Step 2 — Angular Compiler reads the template

When you run *ng serve*, Angular's compiler reads your *.html* file and converts it into JavaScript instructions.

Step 3 — The Component class connects data

Your *app.component.ts* file is the brain. Angular links this class with your HTML template. If your template had `{{ title }}`, Angular replaces it with the value from the class.

Step 4 — Change Detection watches for updates

The moment something changes (e.g. a button click updates a variable), Angular automatically re-checks the template and updates only what changed — without reloading the page.

Step 5 — DOM Renderer writes to index.html

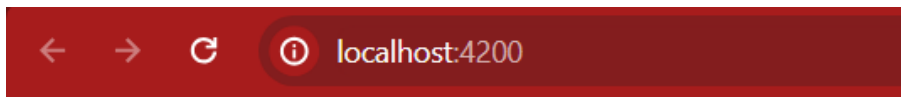
Angular places your component inside *index.html* at the `<app-root></app-root>` tag.

Step 6 — Browser paints the screen

The browser receives the final HTML from Angular and draws it on screen. You see "Hello World".

Example: Component class

```
@Component({
  selector: 'app-root',
  templateUrl: './app.component.html'
})
export class AppComponent {
  title = 'Hello World';
}
```



Hello Angular

The rendered 'Hello Angular' output in the browser

Advantages of Angular

Complete package — Angular gives you everything built-in — routing, forms, HTTP, testing tools. You don't need extra libraries for basic needs.

Auto UI updates — Thanks to data binding and change detection, your screen updates automatically when data changes.

Reusable components — Build a component once and use it anywhere in your app. Saves time and reduces repeated code.

TypeScript catches errors early — Angular uses TypeScript which spots mistakes while you are coding, before the app even runs.

Great for large apps — Angular's organised structure (modules, services, components) keeps big projects clean and easy to manage.

Disadvantages of Angular

Steep learning curve — Angular has many concepts to learn (components, modules, decorators, DI, RxJS). Beginners can feel overwhelmed.

Too much boilerplate code — Even for a small feature you often need multiple files (component, module, service).

Slower for small apps — Angular loads a lot of framework code in the browser. For a simple website, it can feel heavier than needed.

Frequent updates — Angular releases major updates often, which sometimes break older code.

3. Components in Angular

Angular components are the building blocks of Angular applications. They encapsulate a part of the user interface, including its template, styles, and behaviour. Each component represents a reusable piece of UI functionality and can be composed together to create complex applications.

Components follow the principles of **encapsulation**, **reusability**, and **maintainability**.

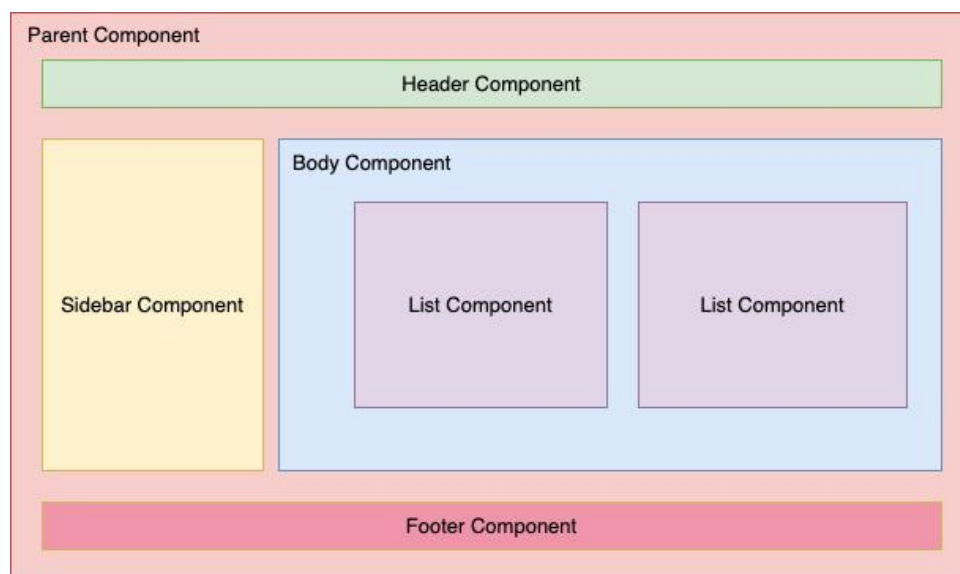
The Three Parts of a Component

Template — Defines the HTML markup of the component's view. Contains placeholders and Angular directives that are replaced with dynamic data at runtime.

Styles — Define the component's visual appearance using CSS rules. Styles can be inline, external files, or preprocessors like Sass.

TypeScript Class — Defines the component's behaviour and logic — properties, methods, and lifecycle hooks.

Component Architecture



Nested component hierarchy: Parent → Header, Body (Sidebar, List × 2), Footer

Structure of an Angular Component

TypeScript (.ts)

```
// hello.component.ts
import { Component } from
 '@angular/core';
@Component({
  selector: 'app-hello',
  templateUrl: './hello.component.html',
  styleUrls: ['./hello.component.css']
})
export class HelloComponent {
  name = 'Susruthi';
}
```

Template & Style

```
<!-- hello.component.html -->
<h1>Hello {{name}}!</h1>

/* hello.component.css */
h1 { color: blue; }
```

Running *ng serve* and opening **http://localhost:4200** will display: **Hello Susruthi!**

Creating a Component with Angular CLI

```
ng generate component component-name
# or shorthand
ng g c component-name
```

The CLI generates four files: a TypeScript class file, an HTML template, a CSS stylesheet, and a spec file for unit testing.

Advantages of Components

Reusability — Create once, use many times. A Navbar component can be used on all pages; a Button component reused everywhere. Saves time and reduces duplicate code.

Easy Maintenance — Each component is independent. If the login page has a bug, fix only LoginComponent — no need to touch the whole app.

Better Code Organisation — Code is divided into small, clearly named folders (navbar/, login/, dashboard/). Clean structure that teams can easily understand.

Easy to Add Features — Add a new ProfileComponent without disturbing existing code. Best for large projects that need to support growth.

Separation of Concerns — HTML handles UI, TypeScript handles logic, CSS handles styling. Clean development and easy to test.

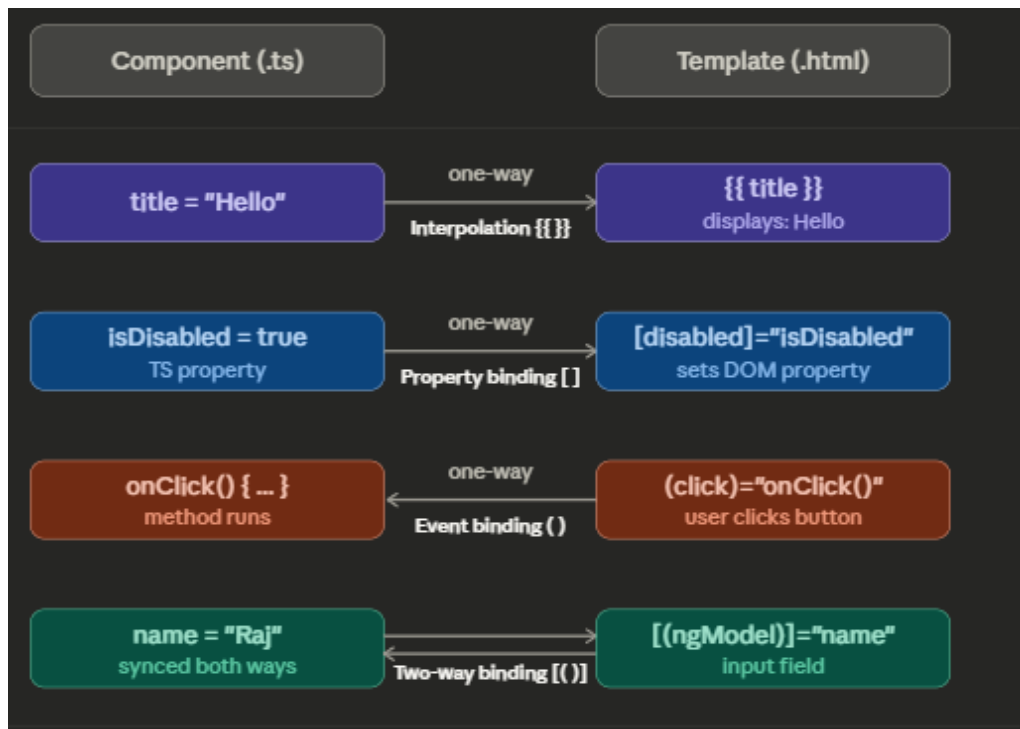
Parallel Development — Teams can work on different components simultaneously — one developer on Navbar, another on Dashboard, another on Forms.

Improved Testing — Test each component individually. Unit testing becomes easy and bugs can be isolated quickly.

Better User Experience — Components update only the required parts of the page — faster UI updates and smooth performance.

4. Data Binding

Data Binding is how your component's TypeScript (logic) and HTML template (view) talk to each other. Think of it like a pipe connecting your code to the screen. Angular supports **four types** of data binding.



Overview of the four data binding types in Angular

1. Interpolation {{ }}

Interpolation is **one-way** binding from Component → Template. You display a TypeScript variable directly inside your HTML using double curly braces.

HTML	TypeScript
<code><h1>{{ name }}</h1></code>	<code>name = 'Susruthi';</code>

2. Property Binding []

Property Binding is **one-way** binding from Component → HTML element property. You bind a component property to a DOM property like `src`, `disabled`, or `style`.

HTML <pre> <!-- Bind src property --> <!-- Bind disabled property --> <button [disabled]="isDisabled">Click Me</button> </pre>	TypeScript <pre> export class AppComponent { imageUrl = 'https://example.com/photo.jpg'; isDisabled = true; btnColor = 'blue'; } </pre>
--	---

3. Event Binding ()

Event Binding is **one-way** binding from Template → Component. You listen to user actions (click, keyup, mouseover) and call a component method.

HTML <pre> <!-- Click event --> <button (click)="onClick()">Click Me</button> <!-- Keyup event --> <input (keyup)="onKeyUp(\$event)" /> <p>{{ message }}</p> </pre>	TypeScript <pre> export class AppComponent { message = ''; onClick() { this.message = 'Button was clicked!'; } onKeyUp(event: any) { this.message = event.target.value; } } </pre>
--	--

4. Two-Way Data Binding [()]

Two-Way Binding combines Property + Event Binding so data flows both ways — Component ↔ Template simultaneously. If data changes in the UI it updates the component; if data changes in the component it updates the UI. Requires **FormsModule** to be imported.

HTML <pre> <input [(ngModel)]="name"> <p>Your name is: {{name}}</p> </pre>	TypeScript <pre> name: string = ''; </pre>
--	--

Data Binding Summary

Type	Syntax	Direction	Use Case
Interpolation (one-way)	<code>{{ value }}</code> e.g. <code><p>{{ name }}</p></code>	Component → View	Display text in HTML e.g. show username, titles, labels
Property Binding (one-way)	<code>[property]</code> e.g. <code>[src]="imageUrl"</code>	Component → View	Set DOM properties e.g. [disabled], [src], [class]
Event Binding (one-way)	<code>(event)</code> e.g. <code>(click)="fn()"</code>	View → Component	Handle user actions e.g. button click, keyup, mouseover
Two-Way Binding (both ways)	<code>[(ngModel)]</code> Needs FormsModule	Component ↔ View	Forms & input fields e.g. login form, search box, settings

Summary table of all four Angular data binding types

5. Directives in Angular

Directives are special instructions in the HTML that tell Angular to do something to the DOM — add, remove, repeat, or change elements. Angular has **three types** of directives.

Component Directives

A Component Directive is the most common type — in fact, every Angular component IS a directive with a template. It has its own HTML template, styles, and logic — essentially a reusable piece of UI.

```
@Component({
  selector: 'app-hello',
  template: '<h1>Hello Angular</h1>'
})
export class HelloComponent { }
```

Structural Directives

Structural Directives change the **structure** of the DOM — they can add, remove, or repeat HTML elements. They are identified by the asterisk ***** before the directive name.

****ngIf — Conditional Rendering***

*ngIf adds or removes an HTML element from the DOM based on a condition. If the condition is true, the element is added; if false, it is completely removed.

HTML	TypeScript
<pre><p *ngIf="isLoggedIn">Welcome!</p> <!-- With else block --> <p *ngIf="isLoggedIn; else notLoggedIn">Welcome!</p> <ng-template #notLoggedIn> <p>Please Login</p> </ng-template></pre>	<pre>// Component isLoggedIn = true;</pre>

****ngFor — Loop / Repeat Elements***

*ngFor repeats an HTML element for each item in a list (array). It loops through the collection and renders one copy of the element per item.

HTML

```
<ul>
<li *ngFor="let fruit of fruits">{{
fruit }}</li>
</ul>
```

TypeScript

```
fruits = ['Apple', 'Mango', 'Banana'];
```

Attribute Directives

Attribute Directives change the **appearance or behaviour** of an existing DOM element without adding or removing it. They are applied like HTML attributes using square brackets `[]`.

Type	Purpose	Example
Component Directive	A directive with a template	<code>@Component</code> itself
Structural Directive	Changes DOM structure	<code>*ngIf</code> , <code>*ngFor</code> , <code>*ngSwitch</code>
Attribute Directive	Changes appearance/behavior	<code>ngClass</code> , <code>ngStyle</code>

Attribute Directives: ngClass, ngStyle, and Custom Directives

ngClass — Add/Remove CSS Classes

ngClass dynamically adds or removes CSS classes on an element based on a condition.

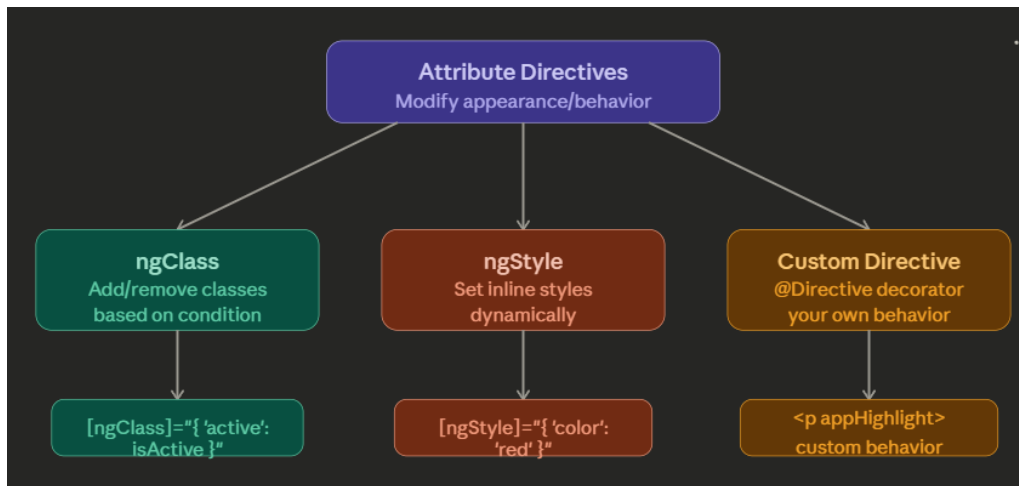
```
<p [ngClass]="{ 'highlight': isActive }">This text may be highlighted</p>
```

ngStyle — Add Inline Styles

ngStyle dynamically sets inline CSS styles on an element using key-value pairs.

```
<p [ngStyle]="{ 'color': textColor }">This text color changes dynamically</p>
```

ngClass vs ngStyle Comparison



Comparison table: ngClass vs ngStyle — syntax, value types, and best use cases

6. Services and Dependency Injection

What is a Service?

A **Service** is a TypeScript class decorated with `@Injectable()` that provides reusable business logic, data access, or utility functions. It is designed to be shared across multiple components using Dependency Injection.

Creating a Service

```
ng generate service user # or shorthand: ng g s user
```

Real-World Uses of Services

- HTTP API calls — Service fetches data from the server
- Authentication logic — Service handles login/logout and stores login status
- Shopping cart data — Service holds cart items so any page can access them
- Logging / error handling — Service silently records errors happening in the app
- Sharing data between components — Service acts as a middleman between unrelated components

Example — Login / Auth Service

```
// auth.service.ts
import { Injectable } from '@angular/core';

@Injectable({ providedIn: 'root' })
export class AuthService {
  isLoggedIn = false;

  login(username: string, password: string) {
    if (username === 'admin' && password === '1234') {
      this.isLoggedIn = true;
      return 'Login Success!';
    }
    return 'Wrong credentials!';
  }

  logout() { this.isLoggedIn = false; }
}
```

```
// login.component.ts (USAGE)
import { AuthService } from '../auth.service';

@Component({ selector: 'app-login', ... })
```

```
export class LoginComponent {
  message = '';
  constructor(private authService: AuthService) {}

  login(user: string, pass: string) {
    this.message = this.authService.login(user, pass);
  }
}
```

Why Do We Need Services?

- Avoid code duplication — write logic once, use it in many components
- Keep components clean — components should only handle UI, not business logic
- Share data between components that are not parent-child
- Make code easier to test and maintain

Dependency Injection (DI)

Dependency Injection means Angular automatically creates and provides a service to your component — you just ask for it in the constructor. Angular creates a **singleton** instance and shares it everywhere.

Feature	ngClass	ngStyle
What it does	Adds/removes CSS classes	Sets inline CSS styles
Syntax	<code>[ngClass]</code>	<code>[ngStyle]</code>
Value type	String / Object / Array	Object (key-value)
Best for	Toggling predefined classes	Dynamic style values
Example	<code>{ 'active': true, 'error': false }</code>	<code>{ 'color': 'red', 'font-size': '16px' }</code>

DI flow: `@Injectable` → Angular Injector stores it → Component asks → Angular delivers same instance

Wrong Way

```
// ■ WITHOUT DI (Wrong way)
export class AppComponent {
  userService: UserService;
  constructor() {
    // Creating service manually – BAD!
    this.userService = new UserService();
    // Each component gets its OWN copy
    // Data is NOT shared
  }
}
```

Correct Angular Way

```
// ■ WITH DI (Correct Angular way)
export class AppComponent {
  // Just ask for it – Angular provides it!
  constructor(private userService:
    UserService) {}
  // Angular creates ONE instance
  // Data IS shared between all
  // components
}
```


7. Angular Modules

A module is like a box that groups related parts of your app together. Every Angular app has at least one module — the **AppModule**.

Basic Syntax

```
@NgModule({
  declarations: [ ], // your components
  imports: [ ], // other modules you need
  providers: [ ], // services
  bootstrap: [ ] // starting component
})
export class AppModule { }
```

The Four Main Properties

declarations — Components and pipes that belong to this module

imports — Other Angular modules you want to use

providers — Services (logic/data) available in this module

bootstrap — The first component to load when the app starts

Types of Modules

Root Module (AppModule) — Every app has exactly one. It is the starting point of the application.

Feature Module — Groups one feature together (e.g., UserModule, ProductModule). Keeps code clean.

Shared Module — Contains components/pipes used in many places. Avoids repeating code.

Core Module — Contains app-wide singleton services. Imported only once in AppModule.

Lazy Loaded Module — Loads only when the user visits a specific route. Makes the app load faster.

8. Forms in Angular

Forms are used to collect input from users — login forms, registration forms, search boxes, etc. Angular provides **two approaches** to building forms.

Template-Driven vs Reactive Forms

Template-Driven

Template-Driven Forms



- Logic lives in the HTML template
- Uses ngModel for two-way binding
- Import FormsModule
- Less TypeScript code
- Good for simple forms (e.g. login)

Reactive

Reactive Forms



- Logic lives in the TypeScript class
- Uses FormGroup, FormControl
- Import ReactiveFormsModule
- More TypeScript code
- Good for complex forms (e.g. registration)

Template-Driven Forms

Step 1 — Import FormsModule

```
// app.module.ts
import { FormsModule } from '@angular/forms';
@NgModule({ imports: [FormsModule] })
```

Step 2 — Use ngModel for Two-Way Binding

ngModel is the heart of template-driven forms. It keeps your HTML input and TypeScript variable in sync.

```
<input [(ngModel)]= "username" name="username" />
<p>Hello, {{ username }}</p>
```

Step 3 — Wrap in <form> with ngForm

Angular automatically attaches ngForm to any <form> tag when FormsModule is imported. Use **#myForm='ngForm'** to access the form's state (valid/invalid, touched, dirty, etc.).

```
<form #myForm="ngForm" (ngSubmit)="onSubmit(myForm)">
  <input [(ngModel)]= "username" name="username" required />
  <button type="submit">Submit</button>
</form>
```

Step 4 — Add Validation

```
<input [(ngModel)]= "email" name="email" required email #emailField="ngModel"
/>
```

```
<span *ngIf="emailField.invalid && emailField.touched">
Please enter a valid email.
</span>
```

Reactive Forms

Reactive Forms are Angular's **code-first** approach. Instead of writing form logic in HTML, you build and control everything in your TypeScript class. TypeScript is in charge.

Step 1 — Import ReactiveFormsModule

```
// app.module.ts
import { ReactiveFormsModule } from '@angular/forms';
@NgModule({ imports: [ReactiveFormsModule] })
```

Step 2 — The Three Building Blocks

FormControl — A single input field (one form control)

FormGroup — A group of fields — the whole form or a section

FormArray — A dynamic list of fields (add/remove at runtime)

Step 3 — Create the Form in TypeScript

```
import { FormGroup, FormControl, Validators } from '@angular/forms';

export class LoginComponent {
  loginForm = new FormGroup({
    name: new FormControl('', [Validators.required, Validators.minLength(3)]),
    email: new FormControl('', [Validators.required, Validators.email]),
  });

  onSubmit() {
    if (this.loginForm.valid) {
      console.log(this.loginForm.value);
    }
  }
}
```

Step 4 — Connect to the HTML Template

```
<form [formGroup]="loginForm" (ngSubmit)="onSubmit()">
<input formControlName="name" placeholder="Your name" />
<input formControlName="email" placeholder="Your email" />
<button type="submit" [disabled]="loginForm.invalid">Submit</button>
</form>
```

Step 5 — Show Validation Messages

```
<input formControlName="name" />
<div *ngIf="loginForm.get('name')?.invalid &&
loginForm.get('name')?.touched">
<span *ngIf="loginForm.get('name')?.errors?.['required']">Name is
required.</span>
<span *ngIf="loginForm.get('name')?.errors?.['minlength']">Minimum 3
characters.</span>
</div>
```

9. Routing and Navigation

Routing allows users to navigate between different components without reloading the page. Angular is a **Single Page Application (SPA)** framework where views are swapped dynamically.

1. Defining Routes

Routes are defined as an array of objects, each mapping a URL path to a component.

```
const routes: Routes = [
  { path: '', component: HomeComponent }, // default
  { path: 'about', component: AboutComponent },
  { path: '**', component: NotFoundComponent } // wildcard
];
```

2. Router Outlet

`<router-outlet>` is a placeholder in the template where the matched component gets rendered.

```
<router-outlet></router-outlet>
```

3. RouterLink

Use **routerLink** instead of *href* to navigate without reloading the page. **routerLinkActive** automatically adds a CSS class to the currently active link.

```
<a routerLink="/about">About</a>
<a routerLink="/about" routerLinkActive="active">About</a>
```

4. Route Parameters

Use route parameters to pass dynamic values through the URL, e.g. */product/5*.

```
// Route definition
{ path: 'product/:id', component: ProductComponent }

// Reading the parameter inside the component
constructor(private route: ActivatedRoute) {}
ngOnInit() {
  const id = this.route.snapshot.paramMap.get('id');
}
```

5. Programmatic Navigation

Navigate from TypeScript code using the **Router** service — useful after form submissions or login.

```
constructor(private router: Router) {}

goToHome() {
  this.router.navigate(['/home']);
}
```

10. HTTP and Observables

Angular provides a built-in **HttpClient** service to communicate with backend servers (REST APIs). It allows you to send and receive data using standard HTTP methods and is part of **HttpClientModule**, which must be imported in *app.module.ts* before use.

Setting Up HttpClient

```
// app.module.ts
import { HttpClientModule } from '@angular/common/http';

@NgModule({
  imports: [HttpClientModule]
})
```

Making HTTP Requests

```
import { HttpClient } from '@angular/common/http';

@Injectable({ providedIn: 'root' })
export class DataService {
  constructor(private http: HttpClient) {}

  getData() {
    return this.http.get('https://api.example.com/items');
  }
}
```

Observables

HttpClient methods return **Observables** (from RxJS). An Observable is a stream of data that you subscribe to in order to receive values over time.

```
// In your component
export class AppComponent implements OnInit {
  items: any[] = [];

  constructor(private dataService: DataService) {}

  ngOnInit() {
    this.dataService.getData().subscribe({
      next: (data: any) => this.items = data,
      error: (err) => console.error(err),
    });
  }
}
```

Common HTTP Methods

GET — Retrieve data from the server

POST — Send new data to the server

PUT — Update existing data on the server

DELETE — Remove data from the server

PATCH — Partially update existing data